```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/di
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

```
spark = SparkSession.builder \
    .appName("Week6_Assignment") \
    .getOrCreate()

print("Spark Session Created Successfully!")
```

```
Spark Session Created Successfully!
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files Sample - Superstore.csv

Sample - Superstore.csv(text/csv) - 2289054 bytes, last modified: 07/06/2026 - 100% done

```
Saving Sample - Superstore.csv to Sample - Superstore (1).csv
```

```
df = spark.read.csv(
    "Sample - Superstore.csv",
    header=True,
    inferSchema=True
)

df.show()
```

```
+------+--------------+----------+----------+--------------+----------
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer I
+------+--------------+----------+----------+--------------+----------
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-1252
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-1252
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-1304
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-2033
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-2033
|     6|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|     7|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|     9|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    10|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    13|CA-2017-114412| 4/15/2017| 4/20/2017|Standard Class|   AA-1048
|    14|CA-2016-161389| 12/5/2016|12/10/2016|Standard Class|   IM-1507
|    15|US-2015-118983|11/22/2015|11/26/2015|Standard Class|   HP-1481
```

```
|    16|US-2015-118983|11/22/2015|11/26/2015|Standard Class|   HP-1481
|    17|CA-2014-105893|11/11/2014|11/18/2014|Standard Class|   PK-1907
|    18|CA-2014-167164| 5/13/2014| 5/15/2014|  Second Class|   AG-1027
|    19|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-2192
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-2192
+------+--------------+----------+----------+--------------+----------
only showing top 20 rows
```

```
df.printSchema()
```

```
root
 |-- Row ID: integer (nullable = true)
 |-- Order ID: string (nullable = true)
 |-- Order Date: string (nullable = true)
 |-- Ship Date: string (nullable = true)
 |-- Ship Mode: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Customer Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- Product Name: string (nullable = true)
 |-- Sales: string (nullable = true)
 |-- Quantity: string (nullable = true)
 |-- Discount: string (nullable = true)
 |-- Profit: double (nullable = true)
```

```
print("Total rows",df.count())
print("Total Columns",len(df.columns))
```

```
Total rows 9994
Total Columns 21
```

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer

Apache Spark follows a master-worker architecture, where different components work together to process large datasets efficiently. The three main components are the Driver, Cluster Manager, and Executors.

1. Driver

The Driver is the main control program of a Spark application. It is responsible for creating the Spark Session, converting user code into execution tasks, scheduling jobs, and coordinating the entire application.

Responsibilities:

Starts the Spark application. Creates the Spark Session. Divides the application into smaller tasks. Sends tasks to the Executors. Collects the results from Executors. Maintains the execution plan (DAG).

Example: When you run:

spark = SparkSession.builder.appName("Week6_Assignment").getOrCreate()

the Driver starts the Spark application and manages all subsequent operations.

2. Cluster Manager

The Cluster Manager is responsible for managing the cluster resources. It allocates CPU cores and memory to the Spark application and launches Executors on the available worker nodes.

Responsibilities:

Allocates system resources. Starts Executors on worker machines. Monitors resource usage. Manages multiple Spark applications running on the cluster.

Examples of Cluster Managers:

Standalone Cluster Manager Hadoop YARN Kubernetes Apache Mesos

3. Executor

Executors are worker processes that perform the actual data processing tasks assigned by the Driver.

Responsibilities:

Execute tasks received from the Driver. Process data stored in partitions. Store intermediate data in memory. Return the execution results to the Driver.

Each Executor runs independently and enables parallel processing, making Spark much faster than traditional processing systems.

Working Flow User Program | ▼ Driver Program | ▼ Cluster Manager | ▼ Executors (Worker Nodes) | ▼ Process Data | ▼ Return Results to Driver Why These Components Are Important The Driver controls the entire Spark application. The Cluster Manager efficiently distributes computing resources. The Executors process data in parallel, improving speed and scalability.

Together, these components allow Spark to process large datasets efficiently.

Brief Insight

Spark's architecture separates application management (Driver), resource allocation (Cluster Manager), and data processing (Executors). This separation enables parallel

execution, efficient resource utilization, and high-performance processing of large-scale data.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer

Lazy Evaluation is a feature of Apache Spark in which transformations are not executed immediately. Instead, Spark records all the transformations (such as filter(), select(), and groupBy()) and waits until an action (such as show(), count(), or collect()) is called. Only then does Spark execute the entire sequence of operations.

Instead of processing each transformation one by one, Spark creates a Directed Acyclic Graph (DAG) of all the operations. It analyzes this execution plan and optimizes it by removing unnecessary computations, combining operations where possible, and reducing data movement (shuffle). This optimization significantly improves the performance of processing large datasets.

For example, if a DataFrame is filtered, selected, and grouped before calling show(), Spark executes all these transformations together in an optimized manner rather than executing each step separately. This reduces execution time, minimizes disk I/O, and uses memory more efficiently.

Advantages of Lazy Evaluation Improves execution speed by optimizing the execution plan. Reduces unnecessary computations. Minimizes disk read/write operations. Reduces shuffle operations whenever possible. Makes Spark highly efficient for large-scale data processing.

Brief Insight

Lazy Evaluation allows Spark to plan first and execute later. By optimizing all transformations before execution, Spark reduces processing time and resource usage, making it much faster than systems that execute every operation immediately.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv(
    "Sample - Superstore.csv",
    header=True,
    inferSchema=True
    )
df.show(5)
```

```
+------+--------------+----------+----------+--------------+----------+
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer I|
```

```
+------+--------------+----------+----------+--------------+----------
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-1252
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-1252
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-1304
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-2033
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-2033
+------+--------------+----------+----------+--------------+----------
only showing top 5 rows
```

Brief Insight

By setting header=True, Spark treats the first row of the CSV file as column names instead of data. Using inferSchema=True automatically detects the correct data types (such as Integer, Double, or String), eliminating the need to manually define the schema. This makes data loading faster, more accurate, and easier for further analysis.

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer

CSV (Comma-Separated Values) is a row-based file format where data is stored one complete row after another. It is simple, human-readable, and widely used for data exchange. However, CSV files are generally larger in size because they do not support built-in compression or store data types, making them slower to read for large datasets.

Parquet is a columnar file format that stores data column by column instead of row by row. It supports built-in compression and preserves the schema (data types). Since Spark often reads only the required columns during analysis, Parquet significantly reduces disk I/O and memory usage, resulting in much faster query execution.

CSV vs Parquet Feature CSV Parquet Storage Format Row-based Columnar File Size Larger Smaller (Compressed) Schema Support No Yes Read Performance Slower Faster Best Use Data exchange Big Data Analytics

Brief Insight

CSV is suitable for sharing and exchanging data because it is easy to read and edit. Parquet is preferred in Apache Spark for large-scale data processing because its columnar storage, compression, and schema support improve performance and reduce storage requirements.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
result = df.filter(col("Category") == "Technology") \
           .select("Product ID", "Sales")

result.show()
```

```
+--------------+--------+
|     Product ID|   Sales|
+--------------+--------+
|TEC-PH-10002275| 907.152|
|TEC-PH-10002033| 911.424|
|TEC-PH-10001949|  213.48|
|TEC-AC-10003027|   90.57|
|TEC-PH-10004977|1097.544|
|TEC-PH-10000486| 371.168|
|TEC-PH-10004093| 147.168|
|TEC-AC-10000171|   45.98|
|TEC-AC-10002167|      45|
|TEC-PH-10003988|    21.8|
|TEC-PH-10002447| 1029.95|
|TEC-AC-10002167|      30|
|TEC-AC-10004633|   13.98|
|TEC-PH-10002726| 167.968|
|TEC-AC-10001998|   19.99|
|TEC-PH-10004093|  73.584|
|TEC-AC-10001767|  95.976|
|TEC-AC-10001552| 238.896|
|TEC-AC-10003499|  74.112|
|TEC-PH-10002844|  27.992|
+--------------+--------+
only showing top 20 rows
```

Brief Insight

The filter() function is used to retrieve only the rows where the Category is Technology. After filtering, the select() function extracts only the Product ID and Sales columns. This reduces unnecessary data processing and improves query efficiency by working only with the required records.

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

```
from pyspark.sql.functions import col
from pyspark.sql.types import DoubleType

# Rename the column and cast Sales to Double

revised_df = df.withColumnRenamed("Product Name", "New_Product_Name")
               .withColumn("Sales", col("Sales").cast(DoubleType()))

# Display the updated schema
revised_df.printSchema()

# Display the first 5 rows
revised_df.select("New_Product_Name", "Sales").show(5)
```

```
root
 |-- Row ID: integer (nullable = true)
 |-- Order ID: string (nullable = true)
 |-- Order Date: string (nullable = true)
 |-- Ship Date: string (nullable = true)
 |-- Ship Mode: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Customer Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- New_Product_Name: string (nullable = true)
 |-- Sales: double (nullable = true)
 |-- Quantity: string (nullable = true)
 |-- Discount: string (nullable = true)
 |-- Profit: double (nullable = true)

+--------------------+--------+
|    New_Product_Name|   Sales|
+--------------------+--------+
|Bush Somerset Col...|  261.96|
|Hon Deluxe Fabric...|  731.94|
|Self-Adhesive Add...|   14.62|
|Bretford CR4500 S...|957.5775|
|Eldon Fold 'N Rol...|  22.368|
+--------------------+--------+
only showing top 5 rows
```

Brief Insight

The withColumnRenamed() function is used to rename an existing column without modifying the original DataFrame, while the cast() function changes the data type of a column. Converting numeric values to the correct data type ensures that mathematical operations such as sum(), avg(), and max() can be performed accurately.

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer

Apache Spark maintains a Lineage Graph, also known as a Directed Acyclic Graph (DAG), which records all the transformations applied to a DataFrame or RDD. Instead of storing multiple copies of intermediate data, Spark keeps track of the sequence of operations used to generate the data.

If a worker node (Executor) fails and some data is lost, Spark does not need to restart the entire application. Instead, it uses the Lineage Graph to identify the missing partitions and recomputes only the lost data by reapplying the required transformations from the original source. This process provides fault tolerance while minimizing recovery time and resource usage.

For example, if a DataFrame is created by reading a CSV file, filtering rows, and selecting specific columns, Spark records each transformation in the DAG. If an Executor fails during processing, Spark recreates only the affected partitions instead of reprocessing the entire dataset.

Advantages of Using the Lineage Graph Provides automatic fault tolerance without manual recovery. Recomputes only the lost partitions instead of the complete dataset. Reduces recovery time after node failures. Ensures reliable and efficient processing of large-scale data. Brief Insight

The Lineage Graph allows Spark to recover from failures by rebuilding only the missing data through previously recorded transformations. This approach makes Spark highly reliable and efficient for distributed data processing without requiring redundant copies of intermediate data.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

```
from pyspark.sql.functions import col, lit, expr
from pyspark.sql.types import DoubleType

filtered_df = df.filter(
    (col("Ship Mode") == "Standard Class") &
    (expr("try_cast(Sales AS DOUBLE)").isNotNull()) &
    (expr("try_cast(Sales AS DOUBLE)") > 1000)
)

filtered_df.show(10)
```

```
+------+--------------+----------+----------+--------------+----------
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer I
+------+--------------+----------+----------+--------------+----------
```

```
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    25|CA-2015-106320| 9/25/2015| 9/30/2015|Standard Class|   EB-1387
|    28|US-2015-150630| 9/17/2015| 9/21/2015|Standard Class|   TB-2152
|    55|CA-2016-105816|12/11/2016|12/17/2016|Standard Class|   JM-1526
|    68|CA-2014-106376| 12/5/2014|12/10/2014|Standard Class|   BS-1159
|   150|CA-2016-114489| 12/5/2016| 12/9/2016|Standard Class|   JE-1616
|   166|CA-2014-139892|  9/8/2014| 9/12/2014|Standard Class|   BM-1114
|   168|CA-2014-139892|  9/8/2014| 9/12/2014|Standard Class|   BM-1114
|   216|CA-2015-146262|  1/2/2015|  1/9/2015|Standard Class|   VW-2177
|   252|CA-2016-145625| 9/11/2016| 9/17/2016|Standard Class|   KC-1654
+------+--------------+----------+----------+--------------+----------
only showing top 10 rows
```

Brief Insight

The filter() function is used to retrieve only the rows that satisfy both conditions. The & (AND) operator ensures that Spark returns records where the Ship Mode is "Standard Class" and Sales is greater than 1000. Applying filters early in a Spark pipeline reduces the amount of data processed in later stages, improving performance.

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer

Predicate Pushdown is a performance optimization technique used by Apache Spark when reading Parquet files. Instead of loading the entire dataset into memory and then applying filter conditions, Spark pushes the filter (predicate) directly to the Parquet file reader. As a result, only the rows and columns that satisfy the specified condition are read from disk.

For example, if a Parquet file contains one million records and the query requests only records where Region = 'West', Spark reads only the matching data instead of scanning the entire file. This significantly reduces the amount of data transferred from disk to memory.

Since Parquet is a columnar storage format, Spark can also read only the required columns, further improving performance. Predicate Pushdown works together with columnar storage to minimize disk I/O, reduce memory consumption, and speed up query execution.

Advantages of Predicate Pushdown Reads only the required data from disk. Reduces memory usage by avoiding unnecessary data loading. Minimizes disk I/O operations. Improves query execution speed. Enhances the performance of Spark applications on large datasets. Brief Insight

Predicate Pushdown makes Spark more efficient by filtering data before it is loaded into memory. Combined with Parquet's columnar storage, it reduces resource usage

and improves the performance of big data processing, especially when working with large datasets.

Q10: Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col

updated_df = df.withColumn(
    "final_price",
    col("Sales") * 1.18
)

updated_df.select("Product Name", "Sales", "final_price").show(10)
```

```
+--------------------+--------+------------------+
|        Product Name|   Sales|       final_price|
+--------------------+--------+------------------+
|Bush Somerset Col...|  261.96|309.11279999999994|
|Hon Deluxe Fabric...|  731.94|          863.6892|
|Self-Adhesive Add...|   14.62|           17.2516|
|Bretford CR4500 S...|957.5775|        1129.94145|
|Eldon Fold 'N Rol...|  22.368|26.394239999999996|
|Eldon Expressions...|   48.86|57.654799999999994|
|          Newell 322|    7.28|            8.5904|
|Mitel 5320 IP Pho...| 907.152|        1070.43936|
|DXL Angle-View Bi...|  18.504|          21.83472|
|Belkin F5C206VTEL...|   114.9|           135.582|
+--------------------+--------+------------------+
only showing top 10 rows
```

Brief Insight

The withColumn() function is used to create a new column in a Spark DataFrame. In this example, the final_price column is calculated by multiplying the Sales value by 1.18, which represents adding an 18% tax. This type of transformation is commonly used in pricing, billing, and financial data processing.

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Answer

In Apache Spark, operations are classified into Transformations and Actions based on how they process data.

Transformations are operations that create a new DataFrame or RDD from an existing one without executing the computation immediately. They are lazy, meaning Spark records these operations and waits until an action is called before executing them. Transformations are used to modify, filter, or organize data.

Examples of Transformations:

filter() – Selects rows that satisfy a given condition. select() – Retrieves only the required columns from a DataFrame.

Actions are operations that trigger the execution of all previously defined transformations. They either return a result to the Driver or write data to storage. When an action is called, Spark executes the optimized execution plan (DAG).

Examples of Actions:

show() – Displays the DataFrame contents on the screen. count() – Returns the total number of rows in the DataFrame. Difference Between Transformations and Actions Feature Transformations Actions Execution Lazy (not executed immediately) Executes immediately Purpose Creates a new DataFrame/RDD Produces output or result Return Type New DataFrame/RDD Value or output Examples filter(), select() show(), count() Brief Insight

Transformations help Spark build an optimized execution plan without performing immediate computation. Actions trigger the execution of that plan and produce the final output, making Spark efficient and fast for processing large datasets.

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
df.write.mode("overwrite").parquet("output_parquet")
```

```
parquet_df = spark.read.parquet("output_parquet")
```

```
from pyspark.sql.functions import col

filtered_df = parquet_df.filter(col("Customer ID").isNotNull())
```

```
filtered_df.show(5)
```

```
+------+--------------+----------+----------+--------------+----------
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer I
+------+--------------+----------+----------+--------------+----------
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-1252
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-1252
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-1304
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-2033
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-2033
+------+--------------+----------+----------+--------------+----------
only showing top 5 rows
```

```
filtered_df.write \
    .mode("overwrite") \
```

```
        .option("header", True) \
        .csv("output_csv")
```

Brief Insight

This example demonstrates a complete Spark data processing pipeline: read → filter → write. The dataset is first loaded from a Parquet file, rows with missing Customer ID values are removed, and the cleaned data is saved as a CSV file. Using Parquet improves performance because it is a columnar file format that supports efficient data reading and filtering.

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer

Apache Spark supports two deployment modes: Client Mode and Cluster Mode. The main difference between them is where the Driver Program runs.

In Client Mode, the Driver Program runs on the machine from which the Spark application is submitted (such as your local computer). The Driver communicates with the Cluster Manager and Executors to schedule and monitor tasks. If the client machine is disconnected or shut down, the Spark application may stop because the Driver is no longer available.

In Cluster Mode, the Driver Program runs inside the cluster on one of the worker nodes. The Cluster Manager is responsible for launching and managing the Driver. Since the Driver is part of the cluster, the application continues to run even if the client disconnects, making Cluster Mode more reliable for long-running production jobs.

Client Mode vs Cluster Mode Feature Client Mode Cluster Mode Driver Location Local machine (client) Inside the cluster Dependency on Client Yes No Reliability Lower Higher Best Use Development and testing Production workloads Brief Insight

Client Mode is best suited for development, debugging, and interactive testing because the Driver runs on the local machine. Cluster Mode is preferred for production environments since the Driver runs within the cluster, providing better reliability, scalability, and fault tolerance for long-running Spark applications.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
from pyspark.sql.functions import col

filtered_df = df.filter(
```

```
    (col("Region") == "North") |
    (col("Category") == "Technology")
)

filtered_df.show(10)
```

```
+------+--------------+----------+----------+--------------+----------
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer I
+------+--------------+----------+----------+--------------+----------
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-1171
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-2192
|    27|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-1394
|    36|CA-2016-117590| 12/8/2016|12/10/2016|   First Class|   GH-1448
|    41|CA-2015-117415|12/27/2015|12/31/2015|Standard Class|   SN-2071
|    42|CA-2017-120999| 9/10/2017| 9/15/2017|Standard Class|   LC-1693
|    45|CA-2016-118255| 3/11/2016| 3/13/2016|   First Class|   ON-1871
|    48|CA-2016-169194| 6/20/2016| 6/25/2016|Standard Class|   LH-1690
|    49|CA-2016-169194| 6/20/2016| 6/25/2016|Standard Class|   LH-1690
+------+--------------+----------+----------+--------------+----------
only showing top 10 rows
```

Brief Insight

The filter() function is used with the OR (|) operator to return rows that satisfy either of the specified conditions. In this example, Spark retrieves records where the Region is "North" or the Category is "Technology". Applying filters helps reduce the amount of data processed in subsequent operations, improving query performance.

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer

In Apache Spark, both .show() and .collect() are actions, but they behave differently when retrieving data.

The .show(5) method displays only the first five rows of a DataFrame. Spark retrieves a small subset of the data and prints it in a tabular format without transferring the entire dataset to the Driver. This makes it a safe and efficient way to inspect large datasets.

On the other hand, .collect() retrieves all the records from the distributed cluster and brings them into the Driver's memory as a Python list. For very large datasets, such as those containing millions or billions of records, this can consume a huge amount of memory, slow down the application, or even cause an OutOfMemoryError, leading to application failure.

Therefore, when exploring or debugging large datasets, it is recommended to use .show(5) or .show(n) instead of .collect().

.show() vs .collect() Feature .show(5) .collect() Data Retrieved Only first 5 rows Entire dataset Memory Usage Very low Very high Suitable for Large Data Yes No Common Use Data preview Small datasets or final results

Brief Insight

The .show(5) method is a safe way to preview data because it retrieves only a small number of records, reducing memory usage. In contrast, .collect() loads the complete dataset into the Driver's memory, which can be risky for large-scale Spark applications.

OVERALL LEARNING

• Learned Spark Architecture.

• Understood Driver, Executors and Cluster Manager.

• Learned Lazy Evaluation.

• Understood DAG execution.

• Worked with CSV and Parquet files.

• Applied filtering and transformations.

• Learned schema handling.

• Built a Spark Data Processing Pipeline.

CONCLUSION

Successfully implemented Spark Architecture concepts using PySpark.

Performed data loading, filtering, transformations, schema handling, and optimized processing techniques.

Compared CSV and Parquet formats and created a complete Spark Data Processing Pipeline.